Chapter 21 — Cheat Sheets and Glossary

This last chapter compresses Chapters 1–20 into pages you can scan in a minute: one-page **cheat sheets** by topic, a **"which tool for which job"** table, and an alphabetical **glossary** of 300+ terms, each pointing to the section where it is explained properly. Nothing here is new — if a line doesn't make sense, follow its section reference (written like **§6.2** = Chapter 6, section 2) back to the full explanation.

Every snippet with a visible result was checked by `code/cheatsheet_checks.py` (Python 3.14; everything shown is valid on 3.12 unless a version is noted).

```
OK  sheet_21_2_types_operators
OK  sheet_21_4_strings
OK  sheet_21_5_methods
OK  sheet_21_6_iteration
OK  sheet_21_7_functions
OK  sheet_21_8_oop
OK  sheet_21_9_exceptions
OK  sheet_21_11_tools
OK  sheet_21_18_copy
OK  sheet_21_19_numpy_pandas
all 10 cheat-sheet checks passed
```

What's inside

#	Sheet	Built from
21.1	Syntax and control flow	Ch 1–3
21.2	Types, numbers and operators	Ch 2
21.3	Data structures and big-O	Ch 5, 6, 17, 19
21.4	String methods and formatting	Ch 4
21.5	List, tuple, dict and set methods	Ch 5, 6
21.6	Comprehensions, iterators and generators	Ch 5, 6, 11
21.7	Functions and arguments	Ch 7
21.8	OOP and dunder methods	Ch 10, 12
21.9	Exceptions	Ch 9
21.10	Files, paths and data formats	Ch 9, 14
21.11	`itertools`, `functools`, `collections`	Ch 6, 7, 11, 12
21.12	Decorators, context managers, metaprogramming	Ch 12
21.13	Type hints	Ch 13
21.14	Standard-library quick picks	Ch 14
21.15	Modules, packages and environments	Ch 8
21.16	Testing and debugging	Ch 15
21.17	Concurrency: which tool?	Ch 16
21.18	Performance and internals	Ch 17
21.19	numpy, pandas, matplotlib essentials	Ch 18
21.20	Interview patterns and gotchas	Ch 19, 20
21.21	Which tool for which job	all
21.22	Glossary (A–Z)	all

Chapter links: 1 · 2 · 3 · 4 · 5 · 6 · 7 · 8 · 9 · 10 · 11 · 12 · 13 · 14 · 15 · 16 · 17 · 18 · 19 · 20

21.1 Syntax and control flow (§1.9–1.11, Ch 3)

Concept	Syntax / rule
Blocks	A line ending in `:` opens a block; its body is indented **4 spaces**; no braces (§1.9)
Comments / docstrings	`# comment` (discarded); `"""Docstring."""` as the first statement → `__doc__` (§1.10)
Line continuation	Implicit inside `()`, `[]`, `{}`; avoid backslashes (§1.9)
Names	Case-sensitive; `snake_case` variables/functions, `CapWords` classes, `UPPER_CASE` constants (§1.11)
Input / output	`input(prompt)` always returns `str`; `print(*values, sep=" ", end="\n")` (§1.7, §1.9)
Script entry point	`if __name__ == "__main__": main()` (§8.5)
`if`	`if c1: … elif c2: … else: …` – first truthy branch wins (§3.2)
Conditional expression	`x = a if cond else b` (§3.3)
`match` (3.10+)	`match cmd:` / `case "quit": …` / `case ["go", direction]: …` / `case {"id": id_}:` / `case Point(x=0):` / `case _:` – no fall-through; bare names **capture** (§3.4)
`for`	`for x in iterable:` – any iterable; `range(start, stop, step)` stop excluded (§3.6)
`while`	`while cond:`; `while True: … break` for loop-and-a-half (§3.7)
Loop control	`break` (leave innermost loop), `continue` (next iteration), `pass` (do nothing) (§3.8)
Loop `else`	Runs only if the loop finished **without** `break` (§3.9)
Walrus	`while (line := f.readline()):` – assign inside an expression, 3.8+ (§2.8)

```
for i, name in enumerate(names, start=1):      # index + value
for a, b in zip(xs, ys, strict=True):          # parallel, error on length mismatch
for k, v in d.items():                          # dict pairs
for x in reversed(seq): ...                     # backwards
for x in sorted(xs, key=len): ...               # sorted view, original unchanged
```

Zen and style: PEP 20 (`import this`) – readability counts, explicit beats implicit, errors shouldn't pass silently. PEP 8 is the style guide; let a formatter (Black/Ruff) apply it (§1.11).

21.2 Types, numbers and operators (Ch 2)

Type	Example	Mutable?	Notes
`int`	`42`, `0b101`, `0xff`, `1_000_000`	no	unbounded precision (§2.5)
`float`	`3.14`, `1e-3`, `float("inf")`	no	IEEE-754 double, ~15–17 significant digits (§2.5)
`complex`	`2 + 3j`	no	`.real`, `.imag`
`bool`	`True`, `False`	no	subclass of `int` (`True + 1 == 2`) (§2.6)
`str`	`"hi"`, `'hi'`, `"""multi"""`, `r"\d"`, `f"{x}"`	no	Unicode text (Ch 4)
`bytes` / `bytearray`	`b"\x00"`	no / yes	raw binary (§4.9)
`list`	`[1, "a"]`	yes	dynamic array (Ch 5)
`tuple`	`(1, 2)`, `1, 2`, `(1,)`	no	the comma makes the tuple (§5.9)
`range`	`range(0, 10, 2)`	no	lazy sequence
`dict`	`{"k": 1}`	yes	insertion-ordered (Ch 6)
`set` / `frozenset`	`{1, 2}`, `set()`	yes / no	`{}` is an empty **`dict`** (§6.6)
`NoneType`	`None`	no	the single "no value" object; test with `is None`

Falsy values: `None`, `False`, `0`, `0.0`, `0j`, `Decimal(0)`, `Fraction(0)`, `""`, `[]`, `()`, `{}`, `set()`, `range(0)`. Everything else is truthy (§2.6).

Conversions (§2.7): `int("42")`, `int("ff", 16)`, `int(3.9) → 3` (truncates), `float("1.5")`, `str(3)`, `bool([]) → False`, `list("ab") → ['a', 'b']`, `ord("A") → 65`, `chr(65) → 'A'`. Bad input raises `ValueError`; wrong type raises `TypeError`.

Operators (§2.8):

Kind	Operators	Watch out
Arithmetic	`+ - * / // % **`	`/` always float; `//` floors (`-7 // 2 == -4`); `%` takes the divisor's sign (`-7 % 3 == 2`); `**` is right-associative and binds tighter than unary minus (`-2 ** 2 == -4`)
Comparison	`== != < <= > >=`, chains `0 < x < 10`	compare floats with `math.isclose`
Identity / membership	`is`, `is not`, `in`, `not in`	`is` only for `None`/singletons
Logical	`and`, `or`, `not`	short-circuit; return an **operand** (`"" or "x" → 'x'`)
Bitwise	`& | ^ ~ << >>`	`~x == -x - 1`
Assignment	`=`, `+=` …, `:=`	`+=` mutates lists in place, rebinds immutables
Built-in maths	`abs`, `round` (half-to-even: `round(2.5) == 2`), `divmod`, `pow(b, e, m)`, `min`, `max`, `sum`	

Money and exactness: `Decimal("0.1")` (from a string!) for money, `Fraction(1, 3)` for exact ratios, `math.isclose` for float comparison (§2.5, §14.8).

21.3 Data structures and big-O (§5.3, §6.2, §6.12, §17.5, §19.1)

Structure	Use for	Fast (avg)	Slow
`list`	ordered, growable sequence	`a[i]`, `a[i]=x`, `append`, `pop()` → O(1)	`insert(0,x)`, `pop(0)`, `x in a`, `remove`, `index` → O(n)
`tuple`	fixed record, dict key	`t[i]` O(1)	no mutation
`dict`	key → value	`d[k]`, `d[k]=v`, `del d[k]`, `k in d` → O(1)	worst case O(n) with collisions
`set`	uniqueness, membership	`add`, `discard`, `x in s` → O(1)	no indexing
`collections.deque`	queue / both-ends stack / last N	`append`, `appendleft`, `pop`, `popleft` → O(1)	middle index O(n)
`heapq` on a list	priority queue, top-k	push/pop O(log n), peek `h[0]` O(1), `heapify` O(n)	search O(n)
`bisect` on a sorted list	ordered search	search O(log n)	`insort` O(n)
`array.array`	compact numbers	O(1) index	Python-level maths not faster
`numpy.ndarray`	numeric arrays	vectorised maths in C	appending is O(n) (copies)

Operation	Cost	Operation	Cost
`sorted(xs)` / `xs.sort()`	O(n log n), O(n) if already sorted, **stable**	`a[i:j]` slice	O(j − i) − copies
`len(x)` (built-ins)	O(1)	`"".join(parts)`	O(total length)
`s1 | s2`	O(len s1 + len s2)	`s1 & s2`	O(min(len s1, len s2))
`min` / `max` / `sum`	O(n)	`Counter(xs)`	O(n)
`heapq.nlargest(k, xs)`	O(n log k)	`list(dict)`	O(n)

Rules: membership tests in a loop → use a `set`; queues → `deque`; repeated "smallest next" → `heapq`; never build strings with `+=` in a loop → `"".join` (§4.8, §17.5).

21.4 String methods and formatting (Ch 4)

Strings are **immutable**: every method returns a new string (§4.2). Index from 0; negative indexes count from the end; slices `s[start:stop:step]` exclude `stop` and never raise (§4.4).

Group	Methods (result for `s = " Hello, World "` where shown)
Case	`lower()`, `upper()`, `title()`, `capitalize()`, `swapcase()`, `casefold()` (aggressive lower for comparisons)
Strip	`strip()` → `'Hello, World'`, `lstrip()`, `rstrip()`, `strip("xy")` removes those *characters*; `removeprefix("ab")`, `removesuffix(".py")` remove an exact prefix/suffix (3.9+)
Split / join	`split()` (any whitespace), `split(",")`, `split(",", 1)`, `rsplit`, `splitlines()`, `partition(":")` → 3-tuple; `", ".join(items)`
Search	`x in s` (preferred test), `find(sub)` → index or `-1`, `index(sub)` → index or `ValueError`, `rfind`, `count(sub)`, `startswith(("a", "b"))`, `endswith`
Replace	`replace(old, new, count)`, `translate(str.maketrans("abc", "xyz"))`, `expandtabs()`
Checks	`isdigit()`, `isdecimal()`, `isnumeric()`, `isalpha()`, `isalnum()`, `isspace()`, `isupper()`, `islower()`, `isidentifier()`
Align	`center(20, "*")`, `ljust(10)`, `rjust(10)`, `zfill(5)` (`"42".zfill(5)` → `'00042'`)
Encode	`s.encode("utf-8")` → `bytes`; `b.decode("utf-8")` → `str` (§4.9)

Escapes: `\n \t \\ \' \" \xhh \uXXXX \N{name}`; raw strings `r"C:\path\n"` keep backslashes (use for regex) (§4.5).

f-string format spec – `{value:[[fill]align][sign][#][0][width][,][.precision][type]}` (§4.6):

Spec	Result	Spec	Result
`f"{3.14159:.2f}"`	`3.14`	`f"{1234567:,}"`	`1,234,567`
`f"{0.256:.1%}"`	`25.6%`	`f"{42:05d}"`	`00042`
`f"{'hi':>6}"`	`hi`	`f"{'hi':*^6}"`	`**hi**`
`f"{255:#x}"` / `f"{255:b}"`	`0xff` / `11111111`	`f"{1234.5:e}"`	`1.234500e+03`
`f"{x=}"` (3.8+)	`x=…`	`f"{obj!r}"`	uses `repr()`

Other formatting: `"{} {}".format(a, b)` for stored templates, `"%s" % x` in `logging` calls, `string.Template` for user-supplied templates, t-strings `t"…"` (3.14) produce a `Template` object for safe processing (§4.6).

Regex (§4.10, §14.3): `re.search` (anywhere), `re.match` (at start), `re.fullmatch` (whole string), `re.findall`, `re.finditer`, `re.sub(pattern, repl, s)`, `re.split`, `re.compile`; groups `(...)`, named `(?P<name>...)`; always raw strings; beware catastrophic backtracking on nested quantifiers.

21.5 List, tuple, dict and set methods (Ch 5, 6)

List (§5.4, §5.5):

Method	Does	Returns
`append(x)`	add one item at the end	`None`
`extend(iterable)`	add each item	`None`
`insert(i, x)`	insert before index i	`None`
`remove(x)`	delete first `x` (`ValueError` if absent)	`None`
`pop(i=-1)`	remove and return item i	the item
`clear()`	empty the list	`None`
`index(x[, start[, end]])`	first position (`ValueError` if absent)	int
`count(x)`	occurrences	int
`sort(key=None, reverse=False)`	sort in place (stable)	`None`
`reverse()`	reverse in place	`None`
`copy()`	shallow copy (= `a[:]`)	new list

Slicing: `a[1:3]`, `a[::-1]`, `a[::2]`; slice assignment `a[1:3] = [9, 9, 9]` can resize; `del a[::2]`. Copies: `=` aliases, `a.copy()` / `a[:]` / `list(a)` shallow, `copy.deepcopy(a)` deep; grids with `[[0] * n for _ in range(m)]`, never `[[0] * n] * m` (§5.8).

Tuple (§5.9–5.13): `count(x)`, `index(x)`; packing `t = 1, 2`; unpacking `a, *rest = t`; `namedtuple("P", "x y")` / `typing.NamedTuple` for named fields (`_asdict()`, `_replace()`).

Dict (§6.1, §6.4):

Method	Does
`d[k]` / `d[k] = v` / `del d[k]`	read (`KeyError` if missing) / insert-or-update / delete
`get(k, default=None)`	read without error
`setdefault(k, default)`	return `d[k]`, inserting `default` first if missing
`update(other)` / `d |= other` / `d1 | d2`	merge (right side wins); `|` returns a new dict (3.9+)
`pop(k[, default])`	remove and return
`popitem()`	remove and return the **last** inserted pair
`keys()`, `values()`, `items()`	live **views**; `keys()` / `items()` support set operations
`dict.fromkeys(keys, value)`	new dict — the *same* `value` object for every key
`copy()`, `clear()`	shallow copy, empty

Iterating a dict yields keys, in insertion order (guaranteed since 3.7). Don't add/remove keys while iterating — loop over `list(d)`.

Set (§6.6–6.8):

Operator	Method	Meaning
`a | b`	`union`	in either
`a & b`	`intersection`	in both
`a - b`	`difference`	in a, not b
`a ^ b`	`symmetric_difference`	in exactly one
`a <= b`, `a < b`	`issubset`	subset, proper subset
`a >= b`	`issuperset`	superset
–	`isdisjoint(b)`	no common items

Mutators: `add(x)`, `update(it)`, `remove(x)` (raises if absent), `discard(x)` (doesn't), `pop()` (arbitrary item), `clear()`, and in-place `|=`, `&=`, `-=`, `^=`. Methods accept any iterable; operators need sets. `frozenset` is the immutable, hashable version.

21.6 Comprehensions, iterators and generators (§5.6, §6.5, Ch 11)

```
[x * x for x in xs]                       # list
[x for x in xs if x > 0]                  # filter: `if` at the END
["pos" if x > 0 else "neg" for x in xs]   # choose: if-else BEFORE `for`
[y for row in grid for y in row]          # nested: same order as the loops
{k: v for k, v in pairs}                  # dict
{w.lower() for w in words}                # set
(x * x for x in xs)                       # generator expression — lazy
sum(x * x for x in xs)                    # parentheses can be dropped as sole argument
```

Concept	Rule
Iterable	has `__iter__` → returns an iterator (list, str, dict, file, range) (§11.1)
Iterator	has `__next__` (raises `StopIteration` at the end) and `__iter__` returning itself; **single-pass**
`for` loop	`it = iter(x)` then `next(it)` until `StopIteration`
Generator function	contains `yield`; calling it returns a generator (an iterator) without running the body (§11.2)
`yield from sub`	delegate to a sub-iterator (§11.3)
`send`, `throw`, `close`	two-way generators (§11.3)
Lazy vs eager	generators/ `map` / `filter` / `zip` / `range` produce on demand → constant memory, early exit (§11.4)
Built-in consumers	`list`, `tuple`, `set`, `dict`, `sum`, `min`, `max`, `any`, `all`, `sorted`, `"".join`, `next(it, default)`

```
def read_records(path):          # a pipeline stage: constant memory
    with open(path, encoding="utf-8") as f:
        for line in f:
            if line.strip():
                yield line.rstrip("\n").split(",")
```

21.7 Functions and arguments (Ch 7)

```
def f(pos_only, /, normal, *args, kw_only, **kwargs) -> None:
    """Docstring."""
```

Part	Meaning
before `/`	positional-only (§7.2)
normal	positional or keyword
`*args`	extra positionals → **tuple**
after `*` or `*args`	keyword-only
`**kwargs`	extra keywords → **dict**
defaults `x=1`	evaluated **once** at `def` time → use `None` for mutable defaults
call-site unpacking	`f(*seq, **mapping)`
return several	`return a, b` returns a tuple; no `return` → `None`

Topic	One-liner
Scope	**LEGB**: Local → Enclosing → Global → Built-in; assignment makes a name local for the whole function; `global x`, `nonlocal x` to rebind outer names (§7.3)
Closure	inner function remembering outer variables; binds **late** (§7.4)
First-class	functions are objects: pass, return, store; `key=` functions (§7.5)
`lambda`	`lambda x: x * 2` — one expression; don't assign to a name
`map` / `filter`	lazy iterators; comprehensions usually clearer (§7.5, §11.7)
Decorator	`@d` over `def f` ≡ `f = d(f)`; use `functools.wraps` (§7.6, §12.1)
Recursion	base case + progress; limit ~1000 frames (`sys.getrecursionlimit()`); no tail-call optimisation (§7.7)
Memoisation	`@functools.cache` / `@lru_cache(maxsize=128)`; arguments must be hashable (§7.6)
Docs and hints	docstring → `help(f)`; annotations are not enforced (§7.8)

21.8 OOP and dunder methods (Ch 10, §12.4–12.6)

```python
from dataclasses import dataclass, field

class Account:
    bank = "PyBank"                       # class attribute (shared)
    def __init__(self, owner: str, balance: float = 0):
        self.owner = owner                # instance attribute
        self._balance = balance           # _ = internal by convention
    @property
    def balance(self) -> float:           # computed/validated attribute
        return self._balance
    @classmethod
    def empty(cls, owner):                # alternative constructor
        return cls(owner)
    @staticmethod
    def valid_amount(x) -> bool:          # helper, no self/cls
        return x > 0
    def __repr__(self):
        return f"Account({self.owner!r}, {self._balance})"

@dataclass(frozen=True, slots=True)
class Point:
    x: float
    y: float = 0.0
    tags: tuple[str, ...] = ()            # mutable defaults need field(default_factory=list)
```

Syntax / built-in	Dunder	Syntax / built-in	Dunder
`Cls(...)` create / init	`__new__` / `__init__`	`repr(x)` / `str(x)`	`__repr__` / `__str__`
`x == y`, `x < y`	`__eq__`, `__lt__` …	`hash(x)`	`__hash__`
`x + y`, `y + x` fallback, `x += y`	`__add__`, `__radd__`, `__iadd__`	`bool(x)`	`__bool__` (else `__len__`)
`len(x)`	`__len__`	`x[k]`, `x[k] = v`, `del x[k]`	`__getitem__`, `__setitem__`, `__delitem__`
`for a in x`	`__iter__` / `__next__`	`a in x`	`__contains__`
`x(...)`	`__call__`	`with x:`	`__enter__` / `__exit__`
`x.attr` missing	`__getattr__`	every `x.attr`	`__getattribute__`
descriptor protocol	`__get__`, `__set__`, `__delete__`	subclass hook	`__init_subclass__`
`f"{x:spec}"`	`__format__`	pattern matching positions	`__match_args__`

Rule	Section
Always write `__repr__`; `__str__` falls back to it	§10.2
Defining `__eq__` without `__hash__` makes instances unhashable	§10.2
Return `NotImplemented` (not `False`) for unsupported operand types	§10.3
`super()` = next class in the instance's MRO (C3 linearisation); `Cls.__mro__`	§10.4, §10.5
Mixins add small capabilities via multiple inheritance	§10.6
Duck typing by default; ABCs enforce methods at instantiation; `Protocol` for structural static typing	§10.7
Prefer composition (has-a) unless it's a genuine is-a	§10.8
`_x` internal by convention; `__x` is name-mangled to `_Cls__x`, not private	§10.9
`@dataclass` options: `frozen`, `order`, `slots`, `kw_only`, `field(default_factory=…)`	§10.10
`enum.Enum`, `IntEnum`, `StrEnum`, `Flag`, `auto()`	§10.11
`__slots__` removes the per-instance `__dict__` → less memory, no new attributes	§10.12
`functools.total_ordering` fills in comparisons from `__eq__` + one of `__lt__` …	§10.2, §12.2

21.9 Exceptions (Ch 9)

```
try:
    value = int(text)
except ValueError as e:               # specific first; the first matching except wins
    log.warning("bad input %r: %s", text, e)
    value = 0
except (TypeError, KeyError):
    raise                             # re-raise unchanged
else:
    print("parsed OK")                # only if no exception
finally:
    cleanup()                         # always

class AppError(Exception): ...        # custom hierarchy
class NotFound(AppError): ...
raise NotFound("user 42") from err    # chain: keeps the cause (__cause__)
e.add_note("while loading config")    # 3.11+
```

Hierarchy (partial)	Raised when
`BaseException` → `SystemExit`, `KeyboardInterrupt`, `GeneratorExit`, `Exception`	root; don't catch it
`Exception` → `ArithmeticError` → `ZeroDivisionError`, `OverflowError`	maths
`LookupError` → `IndexError`, `KeyError`	bad index / missing key
`ValueError` (→ `UnicodeError`)	right type, bad value
`TypeError`	wrong type
`AttributeError`, `NameError` (→ `UnboundLocalError`)	missing attribute / name
`OSError` → `FileNotFoundError`, `PermissionError`, `TimeoutError`, `ConnectionError`	OS / I/O
`ImportError` → `ModuleNotFoundError`	import failed
`RuntimeError` → `RecursionError`, `NotImplementedError`	generic runtime problems
`StopIteration`	iterator exhausted
`ExceptionGroup` + `except*` (3.11+)	several errors at once, e.g. from `asyncio.TaskGroup` (§9.6)

Rules: keep `try` blocks small; never bare `except:` (catches Ctrl+C); prefer EAFP (§9.5); `assert` is for internal invariants only — it is removed by `python -O` (§9.7); read tracebacks bottom-up (§9.1, §15.10).

21.10 Files, paths and data formats (§9.8–9.11, §14.6)

```python
from pathlib import Path
p = Path("data") / "sales.csv"
p.parent.mkdir(parents=True, exist_ok=True)
text = p.read_text(encoding="utf-8");  p.write_text("a,b\n", encoding="utf-8")
with p.open(encoding="utf-8") as f:       # stream big files line by line
    for line in f: ...
with open("img.png", "rb") as f: data = f.read()        # binary: bytes, no encoding
for q in Path(".").rglob("*.py"): print(q.name, q.stem, q.suffix)
```

Mode	Meaning	Mode	Meaning
`"r"`	read text (default)	`"rb"` / `"wb"`	binary read / write
`"w"`	write, **truncates**	`"a"`	append
`"x"`	create, fail if it exists	`"r+"`	read and write

Task	Tool
Paths, globbing, read/write small files	`pathlib.Path` (§9.10)
Copy/move/delete trees, archives	`shutil.copytree`, `shutil.move`, `shutil.rmtree`, `shutil.make_archive`
Scratch files/dirs	`tempfile.TemporaryDirectory()`, `NamedTemporaryFile`
Atomic write	write to a temp file in the same folder, then `os.replace(tmp, final)`
CSV	`csv.reader/writer`, `csv.DictReader/DictWriter`; open with `newline=""`; values come back as `str` (§9.11)
JSON	`json.load/dump` (files), `json.loads/dumps` (strings), `indent=2`, `default=str` for unknown types
Python objects	`pickle` – Python-only, **never load untrusted data**
Tabular analytics	pandas `read_csv` / `read_parquet` (§18.4)
Embedded SQL	`sqlite3` with `?` placeholders (§14.11)

Always pass `encoding="utf-8"` for text files (§9.9).

21.11 `itertools`, `functools`, `collections` (§6.9, §7.6, §11.5, §12.2)

itertools (all lazy) – results for small inputs:

Function	Example → result
`count(start, step)`	`count(10, 5)` → 10, 15, 20, … (infinite)
`cycle(it)` / `repeat(x, n)`	`cycle("ab")` → a, b, a, …; `repeat(0, 3)` → 0, 0, 0
`chain(a, b)` / `chain.from_iterable(xss)`	`chain([1, 2], [3])` → 1, 2, 3
`islice(it, stop)` / `islice(it, start, stop, step)`	`islice(count(), 3)` → 0, 1, 2
`takewhile` / `dropwhile`	`takewhile(lambda x: x < 3, [1, 2, 5, 1])` → 1, 2
`groupby(it, key)`	groups **consecutive** equal keys – sort first
`accumulate(it[, func])`	`accumulate([1, 2, 3])` → 1, 3, 6
`pairwise(it)` (3.10+)	`pairwise("abc")` → ('a','b'), ('b','c')
`batched(it, n)` (3.12+)	`batched("abcde", 2)` → ('a','b'), ('c','d'), ('e',)
`zip_longest(a, b, fillvalue=None)`	pads the shorter input
`product(a, b)` / `product(a, repeat=2)`	Cartesian product
`permutations(it, r)`	`permutations("abc", 2)` → 6 ordered pairs
`combinations(it, r)`	`combinations("abc", 2)` → ab, ac, bc
`combinations_with_replacement`	allows repeats
`starmap(f, pairs)`	`f(*pair)` for each
`tee(it, n)`	n independent iterators from one

functools:

Tool	Use
`@cache` / `@lru_cache(maxsize=…)`	memoise pure functions; `.cache_info()`, `.cache_clear()`
`@cached_property`	compute an attribute once per instance
`@wraps(func)`	keep name/docstring in decorators
`partial(f, *args, **kw)`	pre-fill arguments
`reduce(f, it, initial)`	fold to one value (prefer `sum`, `max`, `math.prod` when they fit)
`@total_ordering`	derive comparison methods
`cmp_to_key(cmp)`	use an old-style comparator as `key=`
`@singledispatch`	overload a function by the first argument's type

collections:

Type	One-liner
`Counter(it)`	counts; `most_common(k)`; missing key → 0; `+ - & |`
`defaultdict(list)`	auto-creates missing values (a *read* inserts)
`deque(maxlen=n)`	O(1) at both ends; `rotate`; bounded buffer
`namedtuple("P", "x y")`	tuple with field names
`OrderedDict`	`move_to_end`, `popitem(last=False)`; order-sensitive `==`
`ChainMap(a, b)`	look up keys in several mappings in order

21.12 Decorators, context managers, metaprogramming (Ch 12)

```python
import functools, time
from contextlib import contextmanager

def timed(func):                                  # plain decorator
    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        t0 = time.perf_counter()
        try:
            return func(*args, **kwargs)
        finally:
            print(f"{func.__name__}: {time.perf_counter() - t0:.3f}s")
    return wrapper

def retry(times):                                 # decorator factory (3 levels)
    def decorator(func):
        @functools.wraps(func)
        def wrapper(*args, **kwargs):
            for attempt in range(times):
                try:
                    return func(*args, **kwargs)
                except OSError:
                    if attempt == times - 1:
                        raise
        return wrapper
    return decorator

@contextmanager
def opened(path):                                 # generator-based context manager
    f = open(path, encoding="utf-8")
    try:
        yield f                                   # value bound by `as`
    finally:
        f.close()                                 # always runs
```

Fact	Section
Stacked decorators are applied bottom-up, and the wrappers run top-down	§12.1
Decorators run at **definition** time; wrappers run at **call** time	§12.1
`__exit__` returning a truthy value suppresses the exception	§12.3
`contextlib` : `suppress` , `closing` , `nullcontext` , `redirect_stdout` , `chdir` (3.11+), `ExitStack` (dynamic number of managers, closes LIFO)	§12.3
Async versions: `async with` , `@asynccontextmanager`	§12.3
Descriptors (`__get__` / `__set__`) power `property` , methods, `classmethod` , `staticmethod` ; data descriptors beat the instance `__dict__`	§12.4
`__getattr__` runs only when normal lookup fails; `__getattribute__` runs for every access	§12.5
Metaclass = class of a class (`type` by default); prefer class decorators or `__init_subclass__`	§12.6
Monkey patching: fine in tests with automatic restore (`mock.patch` , `monkeypatch`); avoid in production	§12.7
Introspection: `type` , `isinstance` , `dir` , `vars` , `getattr` / `setattr` / `hasattr` , `inspect` ; never `eval` / `exec` untrusted input	§12.8

21.13 Type hints (Ch 13)

```python
from typing import Protocol, TypedDict, Literal, Final, NewType, Callable, Any

def greet(name: str, times: int = 1) -> str: ...
scores: dict[str, list[float]] = {}
maybe: int | None = None                        # = Optional[int]
def first[T](xs: list[T]) -> T: ...             # 3.12 generic syntax
type Matrix = list[list[float]]                 # 3.12 type alias statement
Handler = Callable[[str, int], bool]
Mode = Literal["r", "w"]
MAX: Final = 100
UserId = NewType("UserId", int)

class SupportsClose(Protocol):                  # structural: no inheritance needed
    def close(self) -> None: ...

class Movie(TypedDict):
    title: str
    year: int
```

Concept	Meaning	Section
Not enforced	the interpreter ignores hints; mypy/pyright check them statically	§13.1
Gradual typing	add hints where they pay off; unannotated code is `Any`	§13.1
Accept abstract, return concrete	parameters `Iterable` / `Sequence` / `Mapping` (from `collections.abc`), return `list` / `dict`	§13.3
Invariance	`list[bool]` is not a `list[int]`; `Sequence` is covariant	§13.3
Narrowing	`if x is not None:` / `isinstance` refine a union	§13.4
`Any` vs `object`	`Any` switches checking off; `object` allows only what every object supports	§13.5
`TypeVar` / bounds / constraints	generic functions and classes	§13.6
`ParamSpec`	type decorators that preserve signatures	§13.7
`Protocol`, `TypedDict`, `Literal`, `Final`, `NewType`	structural interface, typed dict shape, fixed values, constant, distinct zero-cost type	§13.8–13.10
Tools	`mypy --strict`, `pyright`, `reveal_type(x)`, `# type: ignore[code]`, stub packages `types-*`	§13.13
Runtime validation	pydantic / FastAPI at the boundary	§13.14

21.14 Standard-library quick picks (Ch 14)

Need	Module / call	Section
Dates and times	`datetime.now(UTC)`, `ZoneInfo("Europe/Paris")`, `isoformat()`, `fromisoformat()`, `timedelta` – store aware UTC	§14.1
Durations / timeouts	`time.perf_counter()` / `time.monotonic()`; `time.time()` = epoch seconds	§14.2
Regex	`re` (see 21.4)	§14.3
Logging	`log = logging.getLogger(__name__)`; configure once (`basicConfig` / `dictConfig`); `log.info("x=%s", x)`; `log.exception(...)` in `except`	§14.4
CLI arguments	`argparse.ArgumentParser`, `add_argument`, subcommands	§14.5
Environment, argv, exit	`os.environ.get("KEY", default)`, `sys.argv`, `sys.exit(code)`	§14.6
Run commands	`subprocess.run(["git", "status"], check=True, capture_output=True, text=True, timeout=30)` – a list, not a shell string	§14.6
Random	`random.Random(42)` for reproducible simulations; `secrets.token_urlsafe()` for tokens/passwords	§14.7
Maths / stats	`math.isclose`, `math.gcd`, `math.isqrt`, `math.comb`; `statistics.mean/median/stdev/quantiles`	§14.8
Exact numbers	`decimal.Decimal("0.10")`, `fractions.Fraction(1, 3)`	§14.8
Copy / pretty print	`copy.deepcopy`, `pprint.pp`	§14.9
IDs / hashes	`uuid.uuid4()`; `hashlib.sha256(b).hexdigest()` (use a KDF such as `hashlib.scrypt` for passwords)	§14.9
Embedded DB	`sqlite3.connect`, `with con:` transactions, `?` placeholders	§14.11
URLs / HTTP	`urllib.parse`; `urllib.request` for quick scripts; `requests` / `httpx` for real work	§14.12

21.15 Modules, packages and environments (Ch 8)

Task	Command / rule	Section
Import styles	`import json`, `from pathlib import Path`, `import numpy as np`; avoid `import *`	§8.1
How import works	check `sys.modules` → search `sys.path` → create module, register it, run it once → bind the name	§8.2
Package	folder with `__init__.py` (regular) or without (namespace); `__all__` controls `import *`	§8.3
Relative import	`from .utils import x` – only inside packages; run with `python -m pkg.mod`	§8.4
Script vs module	`if __name__ == "__main__":`	§8.5
Circular import fix	restructure; `import module` instead of `from module import name`; import inside the function	§8.6
Virtual env	`python -m venv .venv` → `source .venv/bin/activate` (Windows: `.venv\Scripts\activate`) → `python -m pip install pkg`	§8.7
Freeze / reproduce	`pip freeze > requirements.txt`, `pip install -r requirements.txt`	§8.8
Modern projects	`pyproject.toml` (metadata, dependencies, tool config) + lock file; `uv add pkg`, `uv run`, `uv sync`; Poetry similar	§8.8
Layout and publishing	`src/` layout; build sdist + wheel (`python -m build` / `uv build`); TestPyPI → PyPI (Trusted Publishing)	§8.9

21.16 Testing and debugging (Ch 15)

```python
# test_prices.py – run with: pytest -q
import pytest
from prices import apply_discount

@pytest.fixture
def cart():
    return {"apple": 1.0, "pear": 2.0}

@pytest.mark.parametrize("pct, expected", [(0, 3.0), (50, 1.5)])
def test_discount(cart, pct, expected):
    assert apply_discount(cart, pct) == pytest.approx(expected)

def test_rejects_negative(cart):
    with pytest.raises(ValueError, match="negative"):
        apply_discount(cart, -5)
```

Tool / idea	Key facts	Section
Test types	unit (one piece, fast), integration (pieces together), end-to-end (whole system); Arrange–Act–Assert	§15.1
`unittest`	`class T(unittest.TestCase)`, `setUp`, `assertEqual`, `assertRaises`; `python -m unittest`	§15.3
pytest	plain `assert`; fixtures (`yield` for teardown, `scope=`), `conftest.py`, `parametrize`, `tmp_path`, `monkeypatch`, `capsys`, `caplog`; marks `skip` / `skipif` / `xfail`	§15.4
pytest CLI	`-q`, `-x` (stop at first failure), `-k expr`, `-m mark`, `--lf` (last failed), `--pdb`	§15.4
Test doubles	dummy, stub, spy, mock, fake; `Mock(return_value=…, side_effect=…)`, `assert_called_once_with`; `patch("where.it.is.used")`; `autospec=True`	§15.5
Coverage	`pytest --cov=pkg --cov-branch --cov-report=term-missing`; coverage ≠ correctness	§15.6
doctest	examples in docstrings run as tests	§15.7
TDD	red → green → refactor	§15.8
Hypothesis	property-based tests: `@given(st.lists(st.integers()))`; shrinks failures	§15.9
Tracebacks	read the last line first, then walk up the frames	§15.10
Debugger	`breakpoint()` → pdb: `n` next, `s` step, `c` continue, `p expr`, `w` where, `u` / `d` up/down, `b` breakpoint, `q` quit	§15.11
Logging vs print	logging for anything that stays in the code	§15.12

21.17 Concurrency: which tool? (Ch 16)

Situation	Use	Why
Tens–hundreds of blocking I/O calls (HTTP, DB, files), sync libraries	`ThreadPoolExecutor` / `threading`	GIL is released while waiting
Thousands of concurrent connections, async libraries	`asyncio`	one thread, cheap tasks
CPU-bound pure Python	`ProcessPoolExecutor` / `multiprocessing`	one GIL per process → real parallelism
CPU-bound numeric work	numpy/pandas vectorisation	C loops
CPU-bound threads with shared memory, compatible deps	free-threaded build (`python3.14t`)	no GIL (optional build)
Pipeline stages	threads/processes + `queue.Queue` (bounded)	backpressure, sentinels for shutdown
Blocking call inside async code	`await asyncio.to_thread(func, ...)`	keeps the event loop responsive
Beyond one machine	Celery, RQ, Dask, Ray, Spark	distributed

	Threads	Processes	asyncio
Parallel CPU (standard build)	no	yes	no
Shared memory	yes (needs locks)	no (pickled messages)	yes (one thread)
Switching	pre-emptive	OS	cooperative at `await`
Cost per unit	~MB stack	whole interpreter	~KB

```
from concurrent.futures import ThreadPoolExecutor, as_completed
with ThreadPoolExecutor(max_workers=8) as ex:
    futures = [ex.submit(fetch, url) for url in urls]
    for fut in as_completed(futures):          # completion order; ex.map keeps input order
        print(fut.result())                    # re-raises the worker's exception

import asyncio
async def main():
    async with asyncio.TaskGroup() as tg:      # 3.11+: structured concurrency
        tasks = [tg.create_task(fetch_async(u)) for u in urls]
    return [t.result() for t in tasks]
asyncio.run(main())
```

Synchronisation (§16.4): `Lock` (mutual exclusion; `with lock:`), `RLock` (re-entrant), `Semaphore(n)` (at most n holders), `Event`, `Condition`. Pitfalls (§16.11): races on shared state, deadlocks (take locks in a global order, use timeouts), blocking calls in coroutines, lost exceptions in fire-and-forget tasks, missing `if __name__ == "__main__":` with processes.

21.18 Performance and internals (Ch 17)

Fact	Section
Everything is an object with identity (`id`), type and value; CPython objects carry a reference count and a type pointer	§17.1
Memory: reference counting frees objects immediately; a generational cyclic GC collects reference cycles; `weakref` refers without owning	§17.2
Small ints −5…256 and some strings are cached/interned → never compare values with `is`	§17.3
Assignment shares; `.copy()` is shallow; `deepcopy` is deep	§17.4

Measure first (§17.6):

Question	Tool
How fast is this snippet?	`python -m timeit "..."`, `timeit.timeit(fn, number=…)`, `%timeit` in Jupyter
Which functions are slow?	`python -m cProfile -s cumtime script.py`, `pstats` (ncalls, tottime, cumtime)
Which lines?	`line_profiler` (`kernprof -l -v`)
Live / low-overhead	`py-spy top --pid N`, `scalene`, `pyinstrument`
Which lines allocate memory?	`tracemalloc` snapshots

Optimise in this order (§17.7): better algorithm/data structure (set for membership, dict for lookup, deque for queues) → cache repeated work (`functools.cache`) → vectorise with numpy/pandas → stream with generators → built-ins, comprehensions, `str.join`, local variables in hot loops → `__slots__` for many small objects → compile hot spots (numba, Cython, Rust/PyO3, mypyc) or use PyPy → parallelise (processes). Newer CPython releases (3.11+) are faster for free.

21.19 numpy, pandas, matplotlib essentials (Ch 18)

numpy (§18.1–18.3):

```
import numpy as np
a = np.array([[1, 2, 3], [4, 5, 6]])       # shape (2, 3), dtype int64
a.shape, a.ndim, a.dtype, a.size
np.zeros((2, 3)); np.ones(3); np.arange(0, 10, 2); np.linspace(0, 1, 5); np.eye(3)
a[0, 1]; a[:, 1]; a[a > 2]                  # index, column slice (view), boolean mask (copy)
a.sum(axis=0)                               # per column -> [5, 7, 9]
a.mean(axis=1)                              # per row    -> [2., 5.]
a * 10; a + np.array([10, 20, 30])          # vectorised + broadcasting
a @ a.T                                     # matrix product
a.reshape(3, 2); a.T; np.concatenate([a, a]); np.vstack; np.hstack
rng = np.random.default_rng(42); rng.normal(size=3); rng.integers(0, 10, 5)
np.linalg.solve(A, b); np.linalg.eigh(S)
```

Rules: slices are **views** (changing them changes the original; `.copy()` to detach); masks and fancy indexing return copies; broadcasting compares shapes from the right – dimensions must be equal or one of them 1; `axis=0` collapses rows (result per column), `axis=1` collapses columns (result per row); vectorise instead of Python loops.

pandas (§18.4–18.11, pandas 3.x):

Task	Code
Load / save	`pd.read_csv(path, parse_dates=["date"])`, `pd.read_parquet`, `df.to_parquet`
First look	`df.head()`, `df.info()`, `df.describe()`, `df["col"].value_counts()`, `df.isna().sum()`
Select	`df["col"]`, `df[["a", "b"]]`, `df.loc[row_labels, cols]` (labels, end **inclusive**), `df.iloc[0:5, 1]` (positions, end exclusive)
Filter	`df[(df.qty > 5) & (df.city == "Oslo")]` (use `&`, `|`, `~` with parentheses), `df.query("qty > 5")`
New columns	`df["rev"] = df.qty * df.price`, `df.assign(rev=lambda d: d.qty * d.price)`, `np.where`, `np.select`, `pd.cut`
Missing data	`isna`, `dropna`, `fillna`, `pd.to_numeric(errors="coerce")`
Group	`df.groupby("city")["rev"].agg(["sum", "mean"])`, named aggregation `agg(total=("rev", "sum"))`, `transform` keeps the original shape
Combine	`pd.merge(a, b, on="id", how="left", validate="m:1", indicator=True)`, `pd.concat([a, b])`
Reshape	`pivot_table(index=, columns=, values=, aggfunc=)`, `melt` (wide → long)
Dates	`pd.to_datetime(col, format=...)`, `.dt.month`, `set_index("date").resample("ME").sum()` (`"ME"` month end in pandas ≥ 2.2/3.0), `rolling(7).mean()`
Sort / top-N	`sort_values("rev", ascending=False)`, `nlargest(3, "rev")`
Speed / memory	vectorise instead of `apply`; `astype("category")` for repeated strings; Parquet over CSV

pandas 3.0 notes (§18.4–18.5): text columns use a dedicated `str` dtype; **Copy-on-Write** is always on → assign with a single `df.loc[mask, "col"] = value`, never chained indexing.

matplotlib (§18.12):

```
import matplotlib
matplotlib.use("Agg")                       # headless scripts
import matplotlib.pyplot as plt
fig, ax = plt.subplots(figsize=(6, 4))      # object-oriented API
ax.plot(x, y, label="sales"); ax.bar(cats, vals); ax.scatter(x, y); ax.hist(data, bins=20)
ax.set(title="Sales", xlabel="Month", ylabel="EUR"); ax.legend()
fig.tight_layout(); fig.savefig("img/sales.png", dpi=120); plt.close(fig)
```

21.20 Interview patterns and gotchas (Ch 19, 20)

Pattern → Python tool (Ch 19):

Signal in the problem	Pattern	Python
sorted array, pair/triple sums	two pointers	indices `lo`, `hi` (§19.2)
contiguous subarray/substring	sliding window / prefix sums	`dict` of last seen, `accumulate`, `defaultdict(int)` (§19.2)
"seen before?", counts, groups	hashing	`set`, `Counter`, `defaultdict(list)` (§19.3)
matching brackets, "next greater"	stack / monotonic stack	`list.append/pop` (§19.4)
FIFO, level order, window max	queue / monotonic deque	`collections.deque` (§19.4)
k largest / smallest, merge k, running median	heap / two heaps	`heapq` (negate for max-heap) (§19.5)
sorted search, minimum feasible value, weighted random pick	binary search	`bisect_left/right`, `lo < hi` template, prefix sums (§19.6)
linked list, tree, serialize a tree	node classes, DFS/BFS	`@dataclass` nodes, pre-order with `#` markers (§19.7)
shortest path unweighted / weighted / dependencies	BFS / Dijkstra / topological sort	`deque`, `heapq`, in-degree counts (§19.8)
dynamic connectivity, cycles, MST	union-find	`parent` list + path compression (§19.9)
optimal value with overlapping subproblems	DP	`@cache` or a table (§19.10)
all combinations / permutations / placements	backtracking	choose → recurse → undo; `path[:]` (§19.11)
bits, parity, powers	bit tricks, number theory	`x & (x-1)`, `pow(b, e, m)`, `math.gcd`, `math.isqrt` (§19.12)
prefix lookups, autocomplete, wildcard, words on a grid	trie	`dict` of children + end-of-word field (§19.13)
locally best choice is provably safe	greedy	one pass after `sorted(key=...)`; justify by exchange argument (§19.14)
merge / insert / overlap ranges, rooms needed	intervals	sort by start (merge) or end (keep most); `heapq` of ends; sweep line (§19.15)
cycle entry, middle, O(1)-space list checks	fast & slow pointers	`slow`, `fast`, compare with `is` (§19.16)
k-th element, three values, sort a linked list	quickselect / Dutch flag / merge sort	random-pivot partition; `low` / `mid` / `high` (§19.17)
spiral/rotate matrix, 32-bit limits, collinear points	math & geometry	boundaries; explicit range check; `gcd`-reduced slopes (§19.18)

Top gotchas (§19.19, §20.2): mutable default args (`None` sentinel) · late-binding closures (`lambda i=i: i`) · `is` vs `==` · int caching/interning · mutating a list/dict while iterating · shallow copies and `[[0]*n]*m` · `dict.fromkeys(k, [])` shares one list; a `defaultdict` read inserts · mutable class attributes shared by instances · circular imports · float precision (`isclose`, `Decimal("…")`) · bare `except:` · shadowing built-ins (`list = …`) · `xs.sort()` returns `None` · `//` floors toward −∞ · recursion limit ~1000.

21.21 Which tool for which job

Job	Reach for	Not	Section
Ordered collection you append to	`list`	`tuple` (immutable)	§5.2
Fixed record / multiple return values / dict key	`tuple`, `NamedTuple`, frozen `dataclass`	`list`	§5.9, §5.13, §10.10
Look up by key	`dict`	list of pairs + search	§6.1
Membership / de-duplication	`set` (`dict.fromkeys` to keep order)	`x in list` in a loop	§6.6
Counting	`Counter`	manual dict with `if k in d`	§6.9
Grouping	`defaultdict(list)`	`setdefault` everywhere	§6.9
Queue / sliding buffer	`deque` (`maxlen`)	`list.pop(0)`	§6.9
Priority queue / top-k	`heapq`	repeated `sort`	§6.10
Sorted inserts and range queries	`bisect` (or `sortedcontainers`)	re-sort after each insert	§6.11
Numeric arrays / maths	NumPy	Python loops over lists	§5.14, §18.1
Tables, group-by, joins	pandas	nested dicts	§18.4
Simple data class	`@dataclass`	hand-written `__init__` / `__repr__` / `__eq__`	§10.10
Validate untrusted input (JSON, API)	pydantic	trusting type hints	§13.14
Named constants / fixed choices	`enum.Enum` / `StrEnum`	bare strings	§10.11
Interface without inheritance	`typing.Protocol`	ABC (when you don't control the classes)	§10.7, §13.8
Reuse behaviour	composition, small mixins	deep inheritance trees	§10.6, §10.8
Cross-cutting behaviour (timing, retry, auth)	decorator	copy-pasted code	§12.1
Setup/teardown that must happen	`with` + context manager	manual `close()`	§9.8, §12.3
Huge or infinite sequences	generators, `itertools`	building full lists	§11.2, §11.5
Paths	`pathlib`	string concatenation, `os.path` sprawl	§9.10
Config / data exchange	JSON (or TOML for config)	pickle	§9.11
Persist Python objects you trust	`pickle` / `joblib`	pickle from strangers	§9.11
Small local database	`sqlite3`	ad-hoc files	§14.11
Diagnostics in running code	`logging`	`print`	§14.4, §15.12
CLI arguments	`argparse` (or click/typer)	parsing `sys.argv` by hand	§14.5
External commands	`subprocess.run([...], check=True)`	`os.system`, `shell=True` with user input	§14.6

Job	Reach for	Not	Section
Tokens / passwords	`secrets`	`random`	§14.7
Money	`Decimal("…")` or integer cents	`float`	§2.5, §14.8
Timestamps	aware `datetime` in UTC + `zoneinfo`	naive local times	§14.1
Durations	`time.perf_counter()`	`time.time()`	§14.2
Tests	pytest (+ fixtures, parametrize)	manual prints	§15.4
Replace a dependency in a test	`unittest.mock.patch` / `monkeypatch`	editing production code	§15.5
Many I/O waits, sync libraries	threads / `ThreadPoolExecutor`	processes	§16.8
Many I/O waits, async libraries	`asyncio`	one thread per connection	§16.9
CPU-heavy pure Python	`ProcessPoolExecutor`	threads (standard build)	§16.7
Find the slow part	`cProfile`, `py-spy`	guessing	§17.6
Memoise a pure function	`functools.cache`	hand-rolled global dict	§7.6
Dependencies per project	`venv` + `pyproject.toml` (+ `uv`)	global `pip install`	§8.7, §8.8
Type-check a codebase	mypy / pyright in CI	runtime `isinstance` everywhere	§13.13

21.22 Glossary (A–Z)

Each entry: **term** — plain meaning — section where it is explained. Links go to the chapter README.

Term	Meaning	Where
ABC (abstract base class)	A class (from `abc`) with `@abstractmethod`s that subclasses must implement before they can be instantiated	10.7
Absolute import	Import by full path from the project/package root (`from pkg.sub import x`)	8.4
Accumulate	`itertools.accumulate`: running totals (or running results of any two-argument function)	11.5
Aliasing	Two names bound to the same object, so a change through one is seen through the other	5.8
Amortised O(1)	An operation that is occasionally expensive but constant on average over many calls (e.g. `list.append`)	5.3
Annotation	A type hint attached to a parameter, return value or variable; stored in `__annotations__`, not enforced	13.2
`Any`	Type-hint escape hatch that disables checking for a value	13.5
argparse	Standard-library module for command-line options, arguments and subcommands	14.5
Argument	The actual value passed at a call (vs the parameter name in the definition)	7.1
`*args`	Parameter collecting extra positional arguments into a tuple	7.2
Arrange–Act–Assert	Test structure: set up, perform the action, check the result	15.1
`array.array`	Compact, typed array of C numbers from the standard library	5.1
`assert`	Statement checking an internal invariant; removed when Python runs with `-O`	9.7
AST (abstract syntax tree)	Tree form of parsed source code that CPython compiles to bytecode	1.8
asyncio	Standard library for single-threaded concurrency with an event loop and `async` / `await`	16.9
Attribute	A name attached to an object (`obj.name`): data or method	10.1
Augmented assignment	`x += 1` and friends; mutates in place for mutable objects that support it	2.8
Autospec	Mock option that makes a mock follow the real object's signature	15.5
Aware datetime	A `datetime` that carries a time zone (`tzinfo`); comparable across zones	14.1
Backpressure	Slowing producers when consumers can't keep up, e.g. with a bounded queue	16.5
Backtracking	Search that builds a candidate step by step and undoes steps that can't succeed	19.11
Banker's rounding	Round-half-to-even, used by `round()` (`round(2.5) == 2`)	2.5
Base case	The input where a recursive function stops recursing	7.7
`BaseException`	Root of all exceptions, including `SystemExit` and `KeyboardInterrupt`; catch `Exception` instead	9.1
BFS (breadth-first search)	Graph/tree traversal level by level with a queue; shortest paths in unweighted graphs	19.8
Big-O	Notation for how cost grows with input size, ignoring constants	17.5
Binary search	Finding a position in sorted data by halving the range each step, O(log n)	19.6
bisect	Standard-library binary search and sorted insertion on lists	6.11
Block	Group of statements under a line ending in `:`, marked by indentation	1.9
Bound method	A function retrieved through an instance, with `self` already filled in	10.1
Branch coverage	Coverage metric that checks both outcomes of each decision were executed	15.6

Term	Meaning	Where
`breakpoint()`	Built-in that drops into the debugger (pdb by default) at that line	15.11
Broadcasting	NumPy rule for combining arrays of different shapes by stretching size-1 dimensions	18.2
Bytecode	The low-level instructions CPython compiles source into and then interprets	1.8
`bytes`	Immutable sequence of raw bytes (0–255); text must be encoded to become bytes	4.9
`cached_property`	Decorator computing an attribute once per instance and storing it	12.4
Call stack	The stack of active function frames; its depth is limited for recursion	7.7
`Callable`	Type hint for function values, e.g. `Callable[[int], str]`	13.7
Capture pattern	A bare name in a `case` pattern that binds the matched value	3.4
Categorical (dtype)	pandas type storing repeated strings as small integer codes — saves memory	18.11
Chained comparison	`a < b < c` meaning `a < b and b < c`, evaluating `b` once	2.8
ChainMap	`collections` mapping that searches several dicts in order	6.9
Circular import	Two modules importing each other so one sees the other half-initialised	8.6
Class	Blueprint that bundles data (attributes) and behaviour (methods)	10.1
Class attribute	Attribute defined in the class body, shared by all instances	10.1
Class decorator	A decorator applied to a class instead of a function	12.1
`classmethod`	Method receiving the class (`cls`) — used for alternative constructors	10.1
Closure	Inner function that remembers variables from its enclosing function	7.4
Code object	Compiled bytecode plus constants and names for one function/module	1.8
Code point	The number Unicode assigns to a character (`ord("A") == 65`)	4.9
Comprehension	Expression building a list/dict/set from an iterable: `[f(x) for x in xs if c]`	5.6
Composition	Building objects that *have* other objects instead of inheriting from them	10.8
Concurrency	Making progress on several tasks by interleaving them (not necessarily at the same instant)	16.1
`concurrent.futures`	High-level executors (`ThreadPoolExecutor`, `ProcessPoolExecutor`) returning futures	16.8
`conftest.py`	pytest file whose fixtures are shared by tests in its directory	15.4
Context manager	Object with `__enter__` / `__exit__` used by `with` to guarantee setup and cleanup	9.8
`contextmanager`	`contextlib` decorator turning a one-`yield` generator into a context manager	12.3
Copy-on-Write	pandas 3 rule: derived objects behave as copies; modify with one `.loc` assignment	18.5
Coroutine	The object returned by calling an `async def` function (a *coroutine function*); the event loop runs it, pausing at each `await`	16.9
Counter	`collections` dict subclass for counting hashable items	6.9
Coverage	Measurement of which lines/branches the tests executed	15.6
cProfile	Built-in deterministic profiler reporting time per function	17.6
CPU-bound	Work limited by computation speed rather than waiting	16.2
CPython	The reference Python implementation, written in C	1.8
Critical section	Code touching shared state that only one thread may run at a time	16.4

Term	Meaning	Where
CSV	Comma-separated text tables; `csv` module; all values read back as strings	9.11
dataclass	`@dataclass` generates `__init__`, `__repr__`, `__eq__` from annotated fields	10.10
DataFrame	pandas 2-D labelled table of columns (each column a Series)	18.4
Deadlock	Threads waiting for each other's locks forever	16.11
`Decimal`	Exact decimal arithmetic type for money; construct from strings	14.8
Decorator	Callable that takes a function (or class) and returns a replacement; `@d` means `f = d(f)`	12.1
Deep copy	Recursive copy of an object and everything it contains (`copy.deepcopy`)	5.8
`defaultdict`	Dict that creates a default value for a missing key on access	6.9
deque	Double-ended queue with O(1) appends/pops at both ends	6.9
Descriptor	Object with `__get__` / `__set__` / `__delete__` that controls attribute access on a class	12.4
DFS (depth-first search)	Traversal that goes as deep as possible before backtracking (recursion or stack)	19.8
dict	Mutable mapping from hashable keys to values; hash table, insertion-ordered	6.1
Dict view	Live, set-like object from `keys()` / `items()` / `values()`	6.4
Dijkstra's algorithm	Shortest paths with non-negative weights using a min-heap	19.8
Docstring	String literal as the first statement of a module/class/function, shown by `help()`	1.10
doctest	Runs examples written in docstrings as tests	15.7
dtype	NumPy/pandas element type (`int64`, `float64`, `bool`, `category`…)	18.1
Duck typing	Using an object by what it can do, not by its class	10.7
Dunder method	"Double underscore" special method like `__len__` that Python calls for syntax	10.2
Dynamic array	Array that over-allocates and grows by copying — how `list` works	5.3
Dynamic programming (DP)	Solving overlapping subproblems once and combining their answers	19.10
Dynamic typing	Types belong to objects; names can be rebound to any type	2.3
EAFP	"Easier to ask forgiveness than permission": try, then handle the exception	9.5
Editable install	Installing a project so source edits take effect without reinstalling (`pip install -e .`)	8.8
Encoding	Rule turning text into bytes (UTF-8 is the standard)	4.9
Enum	Class of named constant members (`enum.Enum`, `StrEnum`, `auto()`)	10.11
`enumerate`	Built-in yielding `(index, item)` pairs	3.6
Epoch	Reference time 1970-01-01 UTC that `time.time()` counts seconds from	14.2
Event loop	The asyncio scheduler that runs ready coroutines and waits for I/O	16.9
Exception	Object representing an error, raised to unwind the stack until handled	9.1
Exception chaining	Linking a new exception to its cause with `raise … from …`	9.4
ExceptionGroup	Exception bundling several errors, handled with `except*` (3.11+)	9.6
Executor	Pool that runs callables on threads or processes and returns futures	16.8
ExitStack	`contextlib` tool managing a dynamic number of context managers	12.3

Term	Meaning	Where
f-string	String literal `f"…{expr:spec}…"` that evaluates and formats expressions	4.6
Fake	Test double with a working lightweight implementation (e.g. in-memory DB)	15.5
Falsy	Value treated as false in a condition: `None`, `False`, zeros, empty containers	2.6
Fast and slow pointers	Two pointers moving at speeds 1 and 2 to find a list's middle or a cycle's entry in O(1) space (Floyd)	19.16
Figure / Axes	matplotlib's whole image / a single plot area within it	18.12
`Final`	Type hint marking a name that must not be reassigned	13.10
First-class function	Functions are objects: they can be stored, passed and returned	7.5
Fixture	pytest function providing setup (and teardown) injected into tests by name	15.4
float	IEEE-754 double-precision number; approximate for most decimals	2.5
Floor division	`//`: divide and round toward negative infinity	2.5
for-else	`else` block that runs when a loop ends without `break`	3.9
Format spec	Mini-language after `:` in f-strings (`>10`, `,.2f`, `%`, `x`)	4.6
`Fraction`	Exact rational number type (`fractions.Fraction(1, 3)`)	14.8
Frame	Per-call workspace holding a function's local variables	7.1
Free-threaded build	Optional CPython build without the GIL (experimental in 3.13, supported in 3.14)	16.6
frozenset	Immutable, hashable set	6.8
functools	Module of higher-order tools: `cache`, `lru_cache`, `partial`, `reduce`, `wraps`	7.6
Future	Placeholder for a result that will be available later (`.result()`)	16.8
Garbage collector (cyclic)	CPython component that finds and frees reference cycles	17.2
Generator	Iterator produced by a function containing `yield`; pauses between values	11.2
Generator expression	Lazy comprehension in parentheses: `(x * x for x in xs)`	11.2
Generic type	Type parameterised by other types (`list[int]`, `dict[str, float]`)	13.3
GIL (Global Interpreter Lock)	CPython lock letting one thread run Python bytecode at a time	16.6
`global`	Statement letting a function rebind a module-level name	7.3
Gradual typing	Adding type hints incrementally; unannotated code is treated as `Any`	13.1
Greedy algorithm	Builds an answer by always taking the locally best choice; correct only when an exchange argument proves it	19.14
groupby (itertools)	Groups *consecutive* items with equal keys – sort first	11.5
groupby (pandas)	Split–apply–combine: aggregate or transform rows per group	18.7
Guard (match)	`if` condition attached to a `case` pattern	3.4
Guard clause	Early `return` / `continue` for special cases to keep code flat	3.2
Hash	Integer computed from a value by `hash()`, used to place it in a hash table	6.2
Hash table	Array of slots indexed by hash – the structure behind dict and set	6.2
Hashable	Has a hash that never changes and consistent `__eq__`; can be a dict key or set item	6.2

Term	Meaning	Where
hashlib	Standard-library cryptographic hash functions (SHA-256…)	14.9
Heap / heapq	Binary min-heap stored in a list; priority queues with O(log n) push/pop	6.10
Higher-order function	Function that takes or returns functions (`map` , `sorted(key=…)` , decorators)	7.5
Hypothesis	Library for property-based testing that generates and shrinks inputs	15.9
I/O-bound	Work that spends most of its time waiting on network, disk or other I/O	16.2
IDE	Integrated development environment (VS Code, PyCharm)	1.6
Identifier	A name for a variable, function, class, etc.	1.9
Identity	Which object a name refers to; compared with `is` , shown by `id()`	2.2
`iloc` / `loc`	pandas selection by integer position / by label (label slices include the end)	18.5
Immortal object	Object (like `None` or small ints) whose refcount never changes, since 3.12	17.1
Immutable	Cannot change after creation (`int` , `str` , `tuple` , `frozenset`)	2.2
Import system	Machinery that finds, loads, caches (`sys.modules`) and binds modules	8.2
Inheritance	A subclass reuses and extends a base class ("is-a")	10.4
Instance	An object created from a class	10.1
Instance attribute	Attribute stored on one object (usually set in `__init__`)	10.1
Integration test	Test of several components working together	15.1
Interning (strings)	Reusing one object for equal strings (identifier-like literals, `sys.intern`)	17.3
Introspection	Examining objects at runtime (`type` , `dir` , `getattr` , `inspect`)	12.8
Invariance	Generic container types don't accept subtypes: `list[bool]` is not `list[int]`	13.3
Iterable	Object that can return an iterator (`__iter__`): anything you can loop over	11.1
Iterator	Object producing items one at a time with `__next__` ; single-pass	11.1
itertools	Standard module of lazy iterator building blocks	11.5
JSON	Text data format; `json` module; safe and portable, limited types	9.11
Keyword	Reserved word of the language (`if` , `def` , `class` …)	1.9
Keyword argument	Argument passed by name (`f(x=1)`)	7.2
Keyword-only parameter	Parameter after `*` or `*args` that must be passed by name	7.2
`**kwargs`	Parameter collecting extra keyword arguments into a dict	7.2
`lambda`	Anonymous one-expression function	7.5
Late binding	Closures look up variables when called, not when created	7.4
Lazy evaluation	Computing values only when requested (generators, `range` , `map`)	11.4
LBYL	"Look before you leap": check conditions before acting	9.5
LEGB	Name lookup order: Local, Enclosing, Global, Built-in	7.3
Linked list	Nodes each pointing to the next; built with small classes in Python	19.7
Linter	Tool that flags bugs and style problems without running code (Ruff, Flake8)	1.6
list	Mutable, ordered sequence of any objects (dynamic array)	5.2
`Literal`	Type hint restricting a value to specific constants	13.10

Term	Meaning	Where
Lock	Synchronisation primitive allowing one thread into a critical section	16.4
Lock file	File pinning exact versions of every dependency for reproducible installs	8.8
Logger / Handler / Formatter	logging's named source / output destination / message layout	14.4
`lru_cache`	Memoising decorator with a size limit, evicting least-recently-used entries	7.6
`match` statement	Structural pattern matching on a subject's shape and values (3.10+)	3.4
Memoization	Caching a function's results by its arguments	7.7
merge (pandas)	SQL-style join of two DataFrames on key columns	18.8
Metaclass	The class of a class (default `type`); controls class creation	12.6
Method	Function defined in a class and called on its instances	10.1
Mixin	Small class adding one capability through multiple inheritance	10.6
Mock	Test double that records calls and returns configured values	15.5
Module	A `.py` file whose names can be imported	8.1
Monkey patching	Replacing attributes of modules/classes at runtime	12.7
Monotonic stack	Stack kept in sorted order to answer "next greater/smaller" in O(n)	19.4
MRO (method resolution order)	Order Python searches classes for attributes, computed by C3 linearisation	10.5
multiprocessing	Running work in separate processes, each with its own interpreter and GIL	16.7
Mutable	Can be changed in place (`list`, `dict`, `set`)	2.2
Mutable default argument	Default like `[]` created once and shared by all calls — use `None`	7.2
mypy	Static type checker for Python type hints	13.13
Naive datetime	A `datetime` without time-zone information	14.1
Name mangling	`__attr` inside a class is renamed to `_Class__attr` to avoid subclass clashes	10.9
namedtuple	Tuple subclass with named fields	5.13
Namespace	Mapping from names to objects (module globals, a function's locals…)	7.3
Namespace package	Package directory without `__init__.py`, possibly spread over several folders	8.3
Narrowing	A type checker refining a union after a check like `is not None`	13.4
ndarray	NumPy's typed, contiguous, n-dimensional array	18.1
`NewType`	Creates a distinct type for checkers with zero runtime cost	13.10
`None`	The single "no value" object; functions without `return` give it	2.4
`nonlocal`	Statement letting an inner function rebind a variable of its enclosing function	7.3
Normalization (Unicode)	Converting equivalent character sequences to one form (NFC/NFD)	4.9
Object	Any value in Python: has identity, type and value	17.1
Open addressing	Hash-collision strategy that probes other slots in the same table	6.2
Operator overloading	Defining dunder methods so operators work on your types	10.3
`Optional[X]`	Type hint meaning `X | None`	13.4
OrderedDict	Dict with `move_to_end` and order-sensitive equality; handy for LRU caches	6.9

Term	Meaning	Where
Package	Directory of modules importable as a unit	8.3
Parallelism	Literally running several computations at the same instant on multiple cores	16.1
Parameter	Name in a function definition that receives an argument	7.1
`ParamSpec`	Type-hint variable capturing a function's full parameter list (for decorators)	13.7
Parametrize	`@pytest.mark.parametrize`: run one test with many inputs	15.4
Parquet	Columnar, compressed file format that keeps dtypes; fast with pandas	18.4
`partial`	`functools.partial`: new callable with some arguments pre-filled	7.6
pathlib	Object-oriented file-system paths (`Path("a") / "b.txt"`)	9.10
pdb	Python's built-in command-line debugger	15.11
PEP	Python Enhancement Proposal — design documents for the language	1.2
PEP 8	Python's style guide	1.11
PEP 20 (Zen of Python)	Aphorisms describing Python's design philosophy (`import this`)	1.11
pickle	Serialises Python objects to bytes; unsafe to load from untrusted sources	9.11
pip	The standard package installer (`python -m pip install …`)	8.7
pivot_table / melt	pandas reshaping: long → wide summary / wide → long	18.9
Positional-only parameter	Parameter before `/` that can't be passed by keyword	7.2
Prefix sum	Running totals that give any range sum in O(1)	19.2
Priority queue	Queue that always serves the smallest (most urgent) item next — `heapq`	6.10
Producer–consumer	Pattern where producers put work on a queue and consumers take it	16.5
Profiling	Measuring where a program spends time or memory	17.6
`property`	Decorator turning methods into a managed attribute (getter/setter)	10.1
Property-based testing	Testing general properties over many generated inputs	15.9
`Protocol`	Structural interface for type checkers — match by methods, not inheritance	13.8
Pydantic	Library validating and parsing data at runtime from type hints	13.14
PyPI	The Python Package Index, where `pip` downloads packages	8.7
pymalloc	CPython's allocator for small objects	17.2
PyPy	Alternative Python implementation with a JIT compiler	1.8
`pyproject.toml`	Standard project file: metadata, dependencies, build system, tool config	8.8
pyright	Fast static type checker (also powers VS Code's Pylance)	13.13
pytest	The de-facto Python testing framework	15.4
Quickselect	Partition-based search for the k-th smallest/largest element, O(n) average	19.17
Race condition	Bug where the result depends on the timing of threads/processes	16.4
Raw string	`r"…"` literal where backslashes are not escapes	4.5
Recursion	A function calling itself on a smaller input	7.7
Recursion limit	Maximum call depth (default 1000), `sys.getrecursionlimit()`	7.7
Reference count	Number of references to an object; freed when it reaches zero	17.2

Term	Meaning	Where
Reflected operator	`__radd__` etc., tried when the left operand doesn't support the operation	10.3
Regression test	Test added so a fixed bug can't silently return	15.1
Regular expression	Pattern language for matching text (`re` module)	4.10
Relative import	Import relative to the current package (`from . import x`)	8.4
REPL	Read-eval-print loop: the interactive `>>>` prompt	1.5
`__repr__` / `__str__`	Developer-facing / user-facing text for an object	10.2
resample	pandas time-series grouping by calendar period (`"ME"`, `"W"`, `"D"`)	18.10
RLock	Re-entrant lock the same thread may acquire several times	16.11
`secrets`	Module for cryptographically secure random tokens	14.7
Semaphore	Lock allowing up to n holders at once	16.4
Sentinel	Special marker value, e.g. `None` default or an end-of-queue object	16.5
Series	pandas 1-D labelled array	18.4
set	Mutable, unordered collection of unique hashable items	6.6
Shallow copy	New outer container sharing the same inner objects	5.8
Short-circuit evaluation	`and` / `or` stop as soon as the result is known	2.8
Slicing	`seq[start:stop:step]` — a new sequence (or view in NumPy)	4.4
Sliding window	Two indices bounding a contiguous range that moves in O(n)	19.2
`__slots__`	Class attribute fixing allowed instance attributes, removing `__dict__`	10.12
Small-int cache	CPython reuses the int objects −5 to 256	17.3
Soft keyword	Word that is a keyword only in context (`match`, `case`, `type`, `_`)	1.9
sqlite3	Standard-library interface to the embedded SQLite database	14.11
src layout	Project layout with code under `src/pkg/` so tests use the installed package	8.9
Stable sort	Equal keys keep their original relative order (Timsort is stable)	5.7
Standard library	Modules shipped with Python ("batteries included")	8.1
`staticmethod`	Method with no implicit first argument; a plain function in the class namespace	10.1
str	Immutable sequence of Unicode code points	4.1
Strong typing	No silent conversion between unrelated types (`"1" + 1` fails)	2.3
Structural typing	Compatibility decided by available methods/attributes, not by class name	13.8
Stub	Test double returning canned answers	15.5
Stub file (`.pyi`)	File with only type signatures for a module	13.13
subprocess	Module for running external commands (`subprocess.run([...])`)	14.6
`super()`	Proxy to the next class in the MRO, used to call overridden methods	10.4
Sweep line	Turning intervals into +1/−1 events sorted by time and keeping a running count	19.15
`sys.modules`	Cache of already-imported modules	8.2
`sys.path`	List of directories searched for imports	8.2

Term	Meaning	Where
t-string	Template string literal `t"…"` (3.14) producing a `Template` object instead of a `str`	4.6
TaskGroup	asyncio structured concurrency: runs tasks and cancels siblings on failure (3.11+)	16.9
TDD (test-driven development)	Red → green → refactor: write a failing test first	15.8
tempfile	Module creating temporary files/directories that clean themselves up	9.10
Test double	Stand-in for a real dependency in tests: dummy, stub, spy, mock, fake	15.5
Thread	Separate flow of execution sharing the process's memory	16.3
timeit	Module for timing small snippets accurately	17.6
Timsort	Python's stable, adaptive merge/insertion sort, O(n log n)	5.7
Topological sort	Ordering of a DAG's nodes so every edge points forward	19.8
tracemalloc	Standard module tracing memory allocations by line	17.6
Traceback	Report of the call stack when an exception is not handled	15.10
Trie	Prefix tree storing strings letter by letter; insert, search and prefix checks are O(word length)	19.13
Truthy	Value treated as true in a condition (anything not falsy)	2.6
try / except / else / finally	Exception-handling statement; `else` on success, `finally` always	9.2
tuple	Immutable ordered sequence; hashable if its items are	5.9
Tuple packing	Commas create a tuple: `t = 1, 2`	5.9
Two pointers	Two indices moving through data to avoid nested loops	19.2
Type alias	Name for a type expression (`type Vector = list[float]`, 3.12)	13.12
Type conversion	Changing a value's type (`int("3")`); implicit only within the numeric tower	2.7
Type hint	Annotation describing expected types for tools and readers	13.1
TypedDict	Type hint for dicts with a fixed set of typed keys	13.9
TypeVar	Placeholder type for generic functions/classes	13.6
ufunc	NumPy universal function operating element-wise in C (`np.sqrt`, `+`)	18.1
Unicode	Standard assigning a number (code point) to every character	4.9
Union type	`A | B`: a value that may be either type	13.4
Union-Find (DSU)	Structure tracking merging groups with near-O(1) find/union	19.9
Unit test	Fast test of one small piece in isolation	15.1
unittest	Standard-library xUnit-style testing framework	15.3
`unittest.mock`	Standard library for mocks and `patch`	15.5
Unpacking	Assigning an iterable's items to several names (`a, *rest = xs`)	5.10
UTF-8	Variable-length Unicode encoding, 1–4 bytes per character; the default to use	4.9
uuid	Module generating universally unique identifiers	14.9
uv	Fast tool managing Python versions, venvs, dependencies and lock files	8.8
Vectorisation	Operating on whole arrays in compiled code instead of Python loops	18.1

Term	Meaning	Where
View (NumPy)	Array sharing memory with another (e.g. a slice)	18.1
Virtual environment (venv)	Isolated per-project interpreter environment and package folder	8.7
Virtual machine (CPython)	The bytecode interpreter loop that executes compiled code	1.8
Walrus operator	`:=` assigns inside an expression (3.8+)	2.8
Weak reference	Reference that doesn't keep an object alive (`weakref`)	17.2
Wheel	Pre-built package archive (`.whl`) that installs without building	8.9
Wildcard pattern	`case _:` – matches anything without binding	3.4
`with` statement	Runs a block inside a context manager's setup and guaranteed cleanup	9.8
`yield`	Produces a value from a generator and pauses it	11.2
`yield from`	Delegates to a sub-generator/iterator	11.3
`zip`	Pairs items from several iterables, stopping at the shortest (`strict=True` to check)	3.6
zoneinfo	Standard-library IANA time zones (`ZoneInfo("Asia/Kolkata")`)	14.1

Chapter TL;DR

- Use the sheets in 21.1–21.19 as a quick reminder; follow the § references for full explanations, examples and pitfalls.
- 21.20 maps interview problem signals to patterns and lists the gotchas that cost the most points.
- 21.21 answers "what should I reach for?" in one table.
- The glossary defines 300+ terms in plain words and points to where each is taught.